Machine Learning Practicals

Complete Python Codes with Outputs

11 practicals | Jupyter-notebook style | scikit-learn, pandas, matplotlib

- All outputs and graphs in this file were generated by actually running the code.
- Only built-in scikit-learn datasets are used, so every cell runs offline with no downloads.
- Required libraries: `pip install numpy pandas matplotlib seaborn scikit-learn`
- Each practical has: Aim, Dataset, Code cells with outputs, Observation and Viva points.
- Run the cells of one practical top to bottom (each practical is independent of the others).

Index

Practical 1: Feature Engineering (Missing Values, Encoding, Scaling) 3

Practical 2: Dimensionality Reduction using PCA and LDA 6

Practical 3: Multiple Linear Regression 9

Practical 4: Polynomial Regression and the Bias-Variance Trade-off 11

Practical 5: Ridge and Lasso Regularization 14

Practical 6: Logistic Regression - Binary Classification 18

Practical 7: Support Vector Machine - Linear and Non-linear Kernels 20

Practical 8: K-Means Clustering - Elbow and Silhouette Methods 22

Practical 9: DBSCAN - Density-Based Clustering 25

Practical 10: Ensemble Learning - Random Forest and Boosting vs Baseline 27

Practical 11: Q-Learning - Grid World 30

Practical 1: Feature Engineering (Missing Values, Encoding, Scaling)

Aim: Apply basic feature engineering - handle missing values, encode categorical variables and scale numerical attributes - to prepare a clean, model-ready dataset.

Dataset: Titanic-style passenger data (small inline sample; real Titanic CSV link given in code)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd

# Titanic-style sample (small, runs without internet).
# For the real dataset use this instead:
# df = pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv")
df = pd.DataFrame({
    "Pclass": [3, 1, 3, 1, 3, 3, 1, 3, 2, 3, 2, 3, 1, 3, 2],
    "Sex": ["male", "female", "female", "female", "male", "male", "male", "female",
            "female", "male", "male", "female", "male", "female", "female"],
    "Age": [22, 38, 26, 35, np.nan, 54, 2, 27, 14, np.nan, 28, 4, 58, 20, np.nan],
    "Fare": [7.25, 71.28, 7.92, 53.10, 8.05, 51.86, 21.07, 11.13, 30.07,
             7.85, 13.00, 16.70, 26.55, 8.05, 26.00],
    "Embarked": ["S", "C", "S", "S", np.nan, "S", "S", "S", "C", "S", "S", "S", "S",
                 np.nan, "S"],
    "Survived": [0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1],
})
print("Shape:", df.shape)
df.head(8)
```

Out[1]:

```
Shape: (15, 6)
```

	Pclass	Sex	Age	Fare	Embarked	Survived
0	3	male	22.0	7.25	S	0
1	1	female	38.0	71.28	C	1
2	3	female	26.0	7.92	S	1
3	1	female	35.0	53.10	S	1
4	3	male	NaN	8.05	NaN	0
5	3	male	54.0	51.86	S	0
6	1	male	2.0	21.07	S	0
7	3	female	27.0	11.13	S	1

In [2]:

```
# Step 1: find the missing values
print(df.isnull().sum())
print("\nData types:")
print(df.dtypes)
```

Out[2]:

```
Pclass      0
Sex         0
Age         3
Fare        0
Embarked    2
Survived    0
dtype: int64

Data types:
Pclass        int64
Sex             str
Age         float64
Fare        float64
Embarked        str
Survived      int64
dtype: object
```

In [3]:

```
# Step 2: handle missing values
df["Age"] = df["Age"].fillna(df["Age"].median())                    # numeric -> median
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])    # category -> mode
print("Missing values after cleaning:", df.isnull().sum().sum())
df
```

Out[3]:

```
Missing values after cleaning: 0
    Pclass     Sex   Age    Fare Embarked  Survived
0        3    male  22.0   7.25        S         0
1        1  female  38.0  71.28        C         1
2        3  female  26.0   7.92        S         1
3        1  female  35.0  53.10        S         1
4        3    male  26.5   8.05        S         0
5        3    male  54.0  51.86        S         0
6        1    male   2.0  21.07        S         0
7        3  female  27.0  11.13        S         1
8        2  female  14.0  30.07        C         1
9        3    male  26.5   7.85        S         0
10       2    male  28.0  13.00        S         0
11       3  female   4.0  16.70        S         1
12       1    male  58.0  26.55        S         1
13       3  female  20.0   8.05        S         1
14       2  female  26.5  26.00        S         1
```

In [4]:

```
# Step 3: encode categorical variables
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})                    # label encoding
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True, dtype=int)  # one-hot
df.head(8)
```

Out[4]:

```
    Pclass  Sex   Age    Fare  Survived  Embarked_S
0       3    0  22.0   7.25         0           1
1       1    1  38.0  71.28         1           0
2       3    1  26.0   7.92         1           1
3       1    1  35.0  53.10         1           1
4       3    0  26.5   8.05         0           1
5       3    0  54.0  51.86         0           1
6       1    0   2.0  21.07         0           1
7       3    1  27.0  11.13         1           1
```

In [5]:

```
# Step 4: scale numerical attributes
from sklearn.preprocessing import StandardScaler, MinMaxScaler

X = df.drop("Survived", axis=1)
y = df["Survived"]
num_cols = ["Age", "Fare"]

X_std = X.copy()
X_std[num_cols] = StandardScaler().fit_transform(X[num_cols])   # mean 0, std 1
X_mm = X.copy()
X_mm[num_cols] = MinMaxScaler().fit_transform(X[num_cols])      # range 0 to 1

print("After StandardScaler:")
print("mean:", X_std[num_cols].mean().round(2).to_dict())
print("std :", X_std[num_cols].std(ddof=0).round(2).to_dict())
print("\nAfter MinMaxScaler:")
print("min:", X_mm[num_cols].min().round(2).to_dict())
print("max:", X_mm[num_cols].max().round(2).to_dict())
```

Out[5]:

```
After StandardScaler:
mean: {'Age': -0.0, 'Fare': -0.0}
std : {'Age': 1.0, 'Fare': 1.0}

After MinMaxScaler:
min: {'Age': 0.0, 'Fare': 0.0}
max: {'Age': 1.0, 'Fare': 1.0}
```

In [6]:

```
# Step 5: final model-ready dataset
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_std, y, test_size=0.2, random_state=42)
print("Train:", X_train.shape, "  Test:", X_test.shape)
X_std.head()
```

Out[6]:

```
Train: (12, 5)   Test: (3, 5)
   Pclass  Sex       Age      Fare  Embarked_S
0       3    0 -0.349699 -0.869357           1
1       1    1  0.733240  2.455511           0
2       3    1 -0.078964 -0.834566           1
```

```
3       1    1  0.530189  1.511483           1
4       3    0 -0.045122 -0.827816           1
```

Observation / Conclusion

Missing values in Age (median) and Embarked (mode) were filled, so the dataset has no nulls. Sex was label-encoded and Embarked one-hot encoded, so every column is numeric. Age and Fare were scaled (StandardScaler gives mean 0 and std 1; MinMaxScaler gives range 0 to 1). The result is a clean, model-ready dataset split into training and test sets.

Viva Quick Points

- Missing values: numeric -> mean/median, categorical -> mode (or drop rows/columns if too many are missing).
- Label encoding gives integers (good for binary/ordinal); one-hot encoding creates one 0/1 column per category (nominal data). drop_first=True avoids the dummy-variable trap.
- StandardScaler: (x - mean)/std. MinMaxScaler: (x - min)/(max - min). Needed for distance/gradient based models (KNN, SVM, PCA, regression).
- Best practice: fit the scaler on the training set only, then transform the test set (avoids data leakage).

Practical 2: Dimensionality Reduction using PCA and LDA

Aim: Apply PCA and LDA on a high-dimensional dataset and observe the effect of feature reduction on model performance and computational efficiency.

Dataset: Digits dataset (sklearn, 1797 images, 64 features, 10 classes)

Code and Output

In [1]:

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

X, y = load_digits(return_X_y=True)      # 8x8 pixel images -> 64 features
print("Dataset shape:", X.shape, "| Classes:", len(np.unique(y)))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

Out[1]:

```
Dataset shape: (1797, 64) | Classes: 10
```

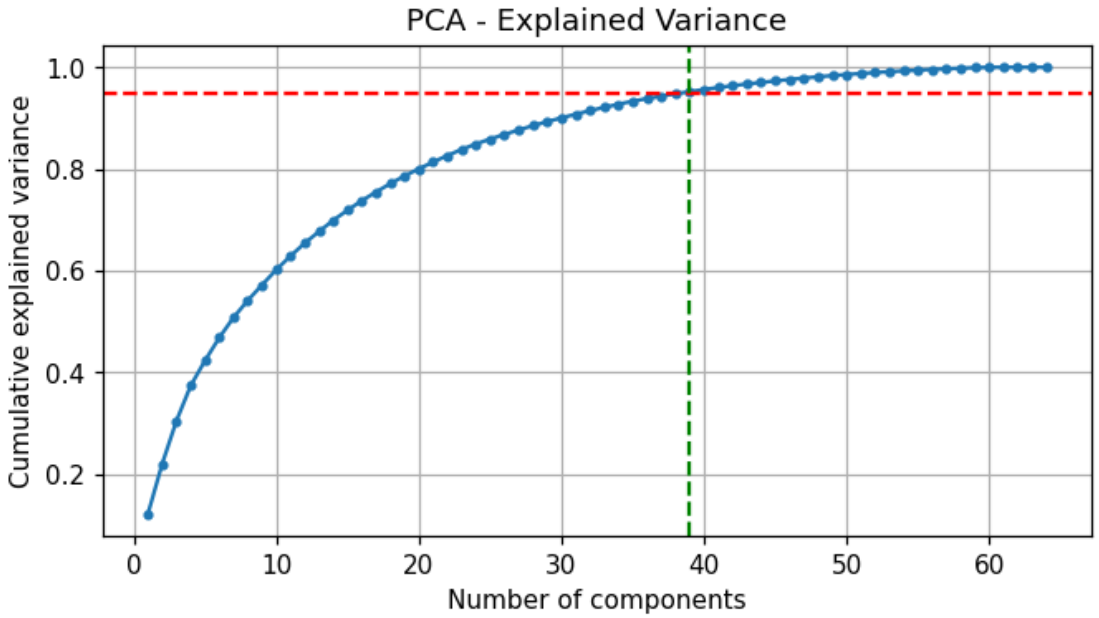
In [2]:

```
# PCA: how many components keep 95% of the variance?
pca_full = PCA().fit(X_train)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
n95 = np.argmax(cum_var >= 0.95) + 1
print("Components needed for 95% variance:", n95)

plt.figure(figsize=(7, 3.5))
plt.plot(range(1, 65), cum_var, marker="o", markersize=3)
plt.axhline(0.95, color="r", linestyle="--")
plt.axvline(n95, color="g", linestyle="--")
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("PCA - Explained Variance")
plt.grid(True)
plt.show()
```

Out[2]:

```
Components needed for 95% variance: 39
```



In [3]:

```
# Compare model performance and training time
def evaluate(name, Xtr, Xte):
    times = []
    for _ in range(5):                          # repeat 5 times, keep the fastest
        start = time.perf_counter()
        model = SVC(kernel="rbf").fit(Xtr, y_train)
        times.append(time.perf_counter() - start)
    acc = accuracy_score(y_test, model.predict(Xte))
    return [name, Xtr.shape[1], round(acc, 4), round(min(times) * 1000, 1)]

results = [evaluate("Original (no reduction)", X_train, X_test)]

pca = PCA(n_components=n95).fit(X_train)
results.append(evaluate(f"PCA ({n95} components)",
                        pca.transform(X_train), pca.transform(X_test)))

pca10 = PCA(n_components=10).fit(X_train)
results.append(evaluate("PCA (10 components)",
                        pca10.transform(X_train), pca10.transform(X_test)))

lda = LDA(n_components=9).fit(X_train, y_train)    # max = classes - 1 = 9
results.append(evaluate("LDA (9 components)",
                        lda.transform(X_train), lda.transform(X_test)))

pd.DataFrame(results, columns=["Method", "Features", "Accuracy", "Fit time (ms)"])
```

Out[3]:

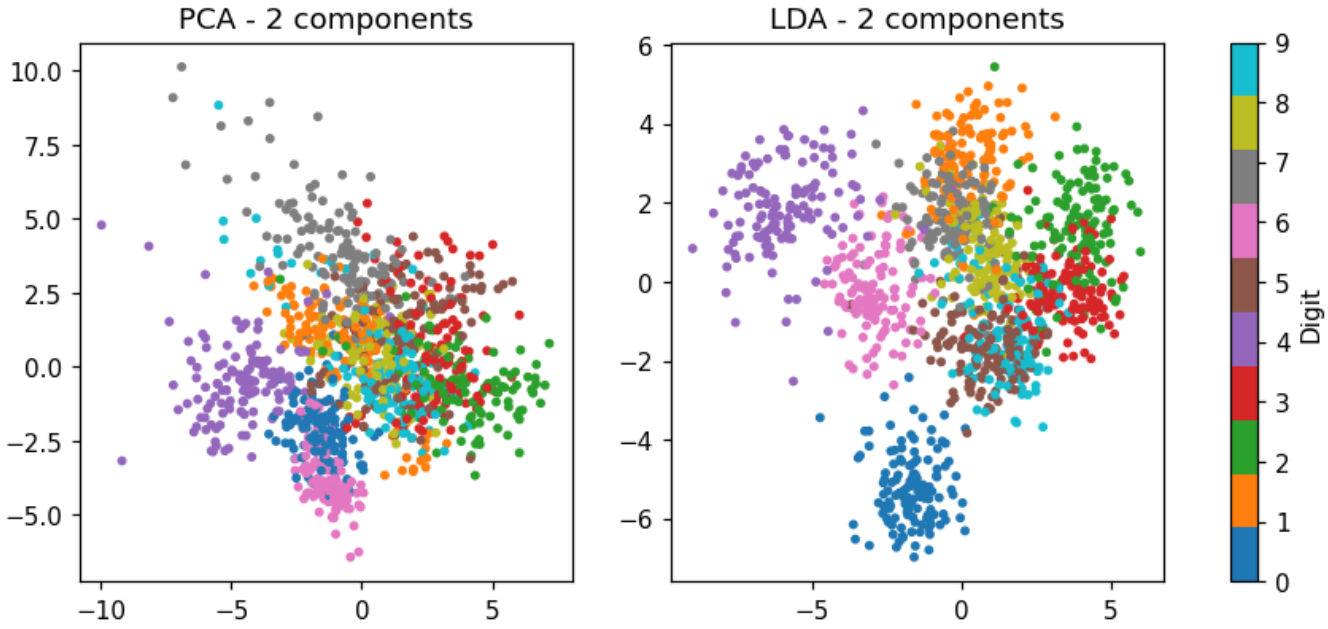
	Method	Features	Accuracy	Fit time (ms)
0	Original (no reduction)	64	0.9833	29.5
1	PCA (39 components)	39	0.9833	32.8
2	PCA (10 components)	10	0.9481	17.8
3	LDA (9 components)	9	0.9630	11.1

In [4]:

```
# Visualise the data in 2-D: PCA (unsupervised) vs LDA (supervised)
X_pca = PCA(n_components=2).fit_transform(X_train)
X_lda = LDA(n_components=2).fit_transform(X_train, y_train)

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
for a, data, title in zip(ax, [X_pca, X_lda], ["PCA - 2 components", "LDA - 2 components"]):
    sc_ = a.scatter(data[:, 0], data[:, 1], c=y_train, cmap="tab10", s=8)
    a.set_title(title)
fig.colorbar(sc_, ax=ax, label="Digit")
plt.show()
```

Out[4]:



Observation / Conclusion

On the Digits data (64 features), PCA needs 39 components to keep 95% of the variance. An RBF-SVM scores 98.3% on all 64 features, 98.3% with 39 PCA components, 94.8% with only 10 PCA components, and 96.3% with just 9 LDA components. Training time drops as the number of features drops (roughly 28 ms for 64 features, 18 ms for 10 PCA components, 11 ms for 9 LDA components; exact times vary per run, and 39 components gave no speed-up on this small dataset). LDA reached high accuracy with very few features because it uses the class labels, whereas PCA at the same size loses more information.

Viva Quick Points

- PCA is unsupervised: finds orthogonal directions of maximum variance. LDA is supervised: finds directions that maximise class separation.
- PCA components <= min(features, samples). LDA components <= (number of classes - 1).
- Fewer features -> faster training and less overfitting, at the cost of some information. Choose components using the cumulative explained-variance plot.
- Always scale the data before PCA.

Practical 3: Multiple Linear Regression

Aim: Develop a multiple linear regression model on a real-world dataset and assess it using MSE, RMSE and R2.

Dataset: Diabetes dataset (sklearn, 442 patients, 10 features, target = disease progression)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

data = load_diabetes(as_frame=True)
df = data.frame
print("Shape:", df.shape)
df.head().round(3)
```

Out[1]:

```
Shape: (442, 11)
     age    sex    bmi     bp     s1     s2     s3     s4     s5     s6  target
0  0.038  0.051  0.062  0.022 -0.044 -0.035 -0.043 -0.003  0.020 -0.018   151.0
1 -0.002 -0.045 -0.051 -0.026 -0.008 -0.019  0.074 -0.039 -0.068 -0.092    75.0
2  0.085  0.051  0.044 -0.006 -0.046 -0.034 -0.032 -0.003  0.003 -0.026   141.0
3 -0.089 -0.045 -0.012 -0.037  0.012  0.025 -0.036  0.034  0.023 -0.009   206.0
4  0.005 -0.045 -0.036  0.022  0.004  0.016  0.008 -0.003 -0.032 -0.047   135.0
```

In [2]:

```
# Split features (X) and target (y), then train / test split
X = df.drop("target", axis=1)
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

Out[2]:

```
Train: (353, 10)  Test: (89, 10)
```

In [3]:

```
# Train the multiple linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept:", round(model.intercept_, 2))
print("\nCoefficients:")
print(pd.Series(model.coef_, index=X.columns).round(2))
```

Out[3]:

```
Intercept: 151.35

Coefficients:
age     37.90
sex   -241.96
bmi    542.43
bp     347.70
s1    -931.49
s2     518.06
s3     163.42
s4     275.32
s5     736.20
s6      48.67
dtype: float64
```

In [4]:

```
# Predict and evaluate
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print("MSE  :", round(mse, 2))
print("RMSE :", round(rmse, 2))
print("MAE  :", round(mae, 2))
print("R2   :", round(r2, 4))
print("Train R2:", round(model.score(X_train, y_train), 4))
```

Out[4]:

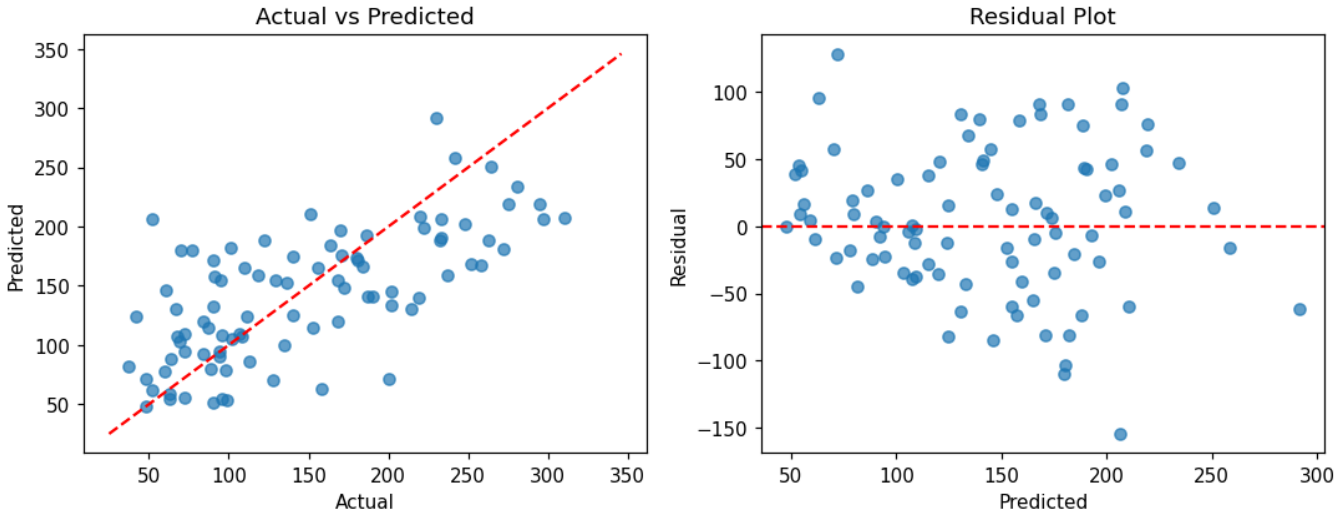
```
MSE  : 2900.19
RMSE : 53.85
MAE  : 42.79
R2   : 0.4526
Train R2: 0.5279
```

In [5]:

```
# Actual vs predicted and residual plots
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].scatter(y_test, y_pred, alpha=0.7)
ax[0].plot([y.min(), y.max()], [y.min(), y.max()], "r--")
ax[0].set_xlabel("Actual"); ax[0].set_ylabel("Predicted")
ax[0].set_title("Actual vs Predicted")

ax[1].scatter(y_pred, y_test - y_pred, alpha=0.7)
ax[1].axhline(0, color="r", linestyle="--")
ax[1].set_xlabel("Predicted"); ax[1].set_ylabel("Residual")
ax[1].set_title("Residual Plot")
plt.tight_layout()
plt.show()
```

Out[5]:



Observation / Conclusion

The multiple linear regression model gave MSE = 2900.19, RMSE = 53.85, MAE = 42.79 and R2 = 0.4526 on the test set (train R2 = 0.5279). The model explains about 45% of the variance in disease progression. Train and test R2 are close, so the model is not overfitting; the moderate R2 shows that a linear model on these 10 features cannot capture everything.

Viva Quick Points

- Model: y = b0 + b1*x1 + b2*x2 + ... + bn*xn. Coefficients are found by minimising the sum of squared errors (least squares).
- MSE = mean((y - y_pred)^2); RMSE = sqrt(MSE) (same units as target); R2 = 1 - SSres/SStot (fraction of variance explained, 1 is perfect).
- Assumptions: linearity, independent errors, constant variance (homoscedasticity), no multicollinearity, normal residuals.

Practical 4: Polynomial Regression and the Bias-Variance Trade-off

Aim: Demonstrate polynomial regression with varying degrees and observe the effect of model complexity on accuracy (underfitting vs overfitting).

Dataset: Diabetes dataset (sklearn) - BMI feature vs disease progression

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True, as_frame=True)
X = X[["bmi"]]                                  # one feature, so we can plot the curve
X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=40, random_state=3)         # small training set shows overfitting
print("Train size:", len(X_train), " Test size:", len(X_test))
```

Out[1]:

```
Train size: 40  Test size: 402
```

In [2]:

```
# Fit polynomial models of different degrees
degrees = [1, 2, 3, 5, 8, 12]
rows = []
models = {}
for d in degrees:
    model = make_pipeline(StandardScaler(), PolynomialFeatures(d), LinearRegression())
    model.fit(X_train, y_train)
    models[d] = model
    tr_mse = mean_squared_error(y_train, model.predict(X_train))
    te_mse = mean_squared_error(y_test, model.predict(X_test))
    te_r2 = r2_score(y_test, model.predict(X_test))
    rows.append([d, round(tr_mse, 1), round(te_mse, 1), round(te_r2, 3)])

pd.DataFrame(rows, columns=["Degree", "Train MSE", "Test MSE", "Test R2"])
```

Out[2]:

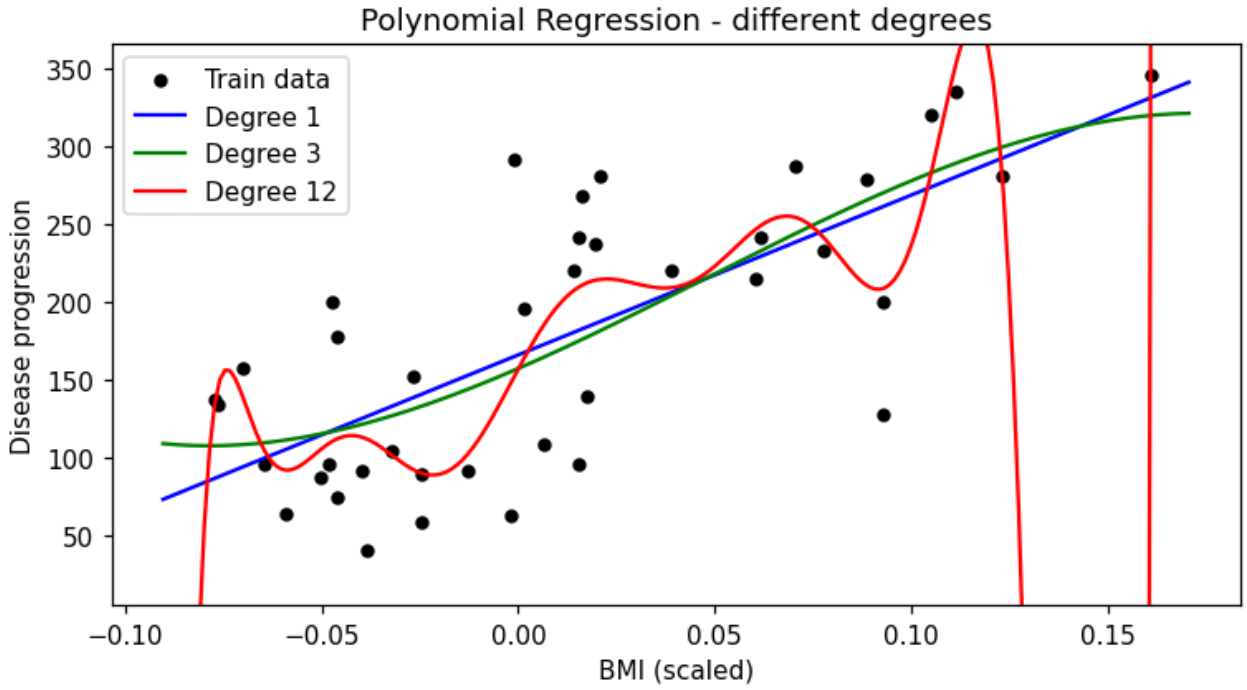
	Degree	Train MSE	Test MSE	Test R2
0	1	3557.7	4153.2	0.272
1	2	3539.7	4119.0	0.278
2	3	3488.0	4129.4	0.276
3	5	3134.3	4570.9	0.199
4	8	2888.6	6085.9	-0.067
5	12	2573.1	1894829.0	-331.091

In [3]:

```
# Plot the fitted curves
x_line = pd.DataFrame({"bmi": np.linspace(X["bmi"].min(), X["bmi"].max(), 200)})

plt.figure(figsize=(8, 4))
plt.scatter(X_train, y_train, color="black", s=20, label="Train data")
for d, c in zip([1, 3, 12], ["blue", "green", "red"]):
    plt.plot(x_line["bmi"], models[d].predict(x_line), color=c, label=f"Degree {d}")
plt.ylim(y.min() - 20, y.max() + 20)
plt.xlabel("BMI (scaled)"); plt.ylabel("Disease progression")
plt.title("Polynomial Regression - different degrees")
plt.legend()
plt.show()
```

Out[3]:



In [4]:

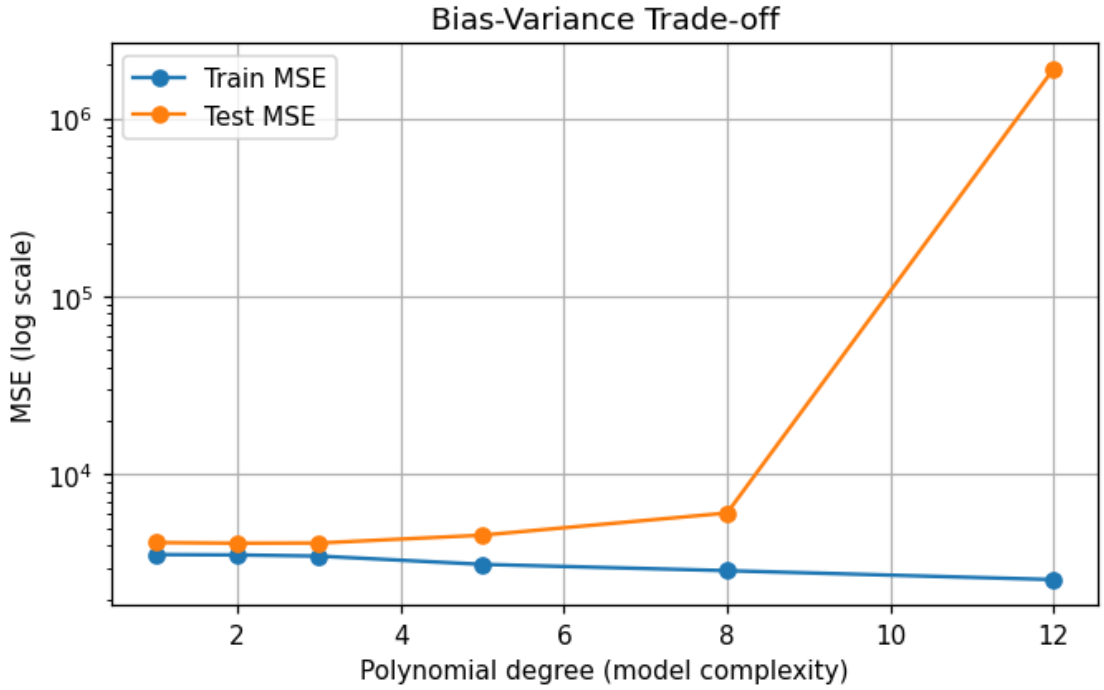
```
# Bias-variance: train error vs test error
tr = [r[1] for r in rows]
te = [r[2] for r in rows]

plt.figure(figsize=(7, 4))
plt.plot(degrees, tr, marker="o", label="Train MSE")
plt.plot(degrees, te, marker="o", label="Test MSE")
plt.yscale("log")                    # log scale: very high degrees blow up
plt.xlabel("Polynomial degree (model complexity)")
plt.ylabel("MSE (log scale)")
plt.title("Bias-Variance Trade-off")
plt.legend()
plt.grid(True)
plt.show()

best = degrees[int(np.argmin(te))]
print("Best degree (lowest test MSE):", best)
```

Out[4]:

```
Best degree (lowest test MSE): 2
```



Observation / Conclusion

Degree 1 is too simple (high bias) and degree 2 gives the lowest test MSE. As the degree increases, the training error keeps falling but the test error rises sharply (degree 12 has a huge test MSE), which is overfitting (high variance). The best model balances bias and variance at a low degree, so more complexity does not always mean better predictions.

Viva Quick Points

- Polynomial regression is still linear regression - on features x, x^2, x^3, ... (PolynomialFeatures does the expansion).
- Low degree -> high bias (underfitting): both train and test error are high.
- Very high degree -> high variance (overfitting): train error is very low, test error rises.
- Best model sits at the lowest test error. Remedies for overfitting: lower degree, more data, regularization (Ridge/Lasso), cross-validation.

Practical 5: Ridge and Lasso Regularization

Aim: Use Ridge and Lasso regularization on a multivariate dataset and observe their influence on overfitting and multicollinearity (regression and classification).

Dataset: Diabetes dataset (regression) and Breast Cancer dataset (classification) from sklearn

Code and Output

In [1]:

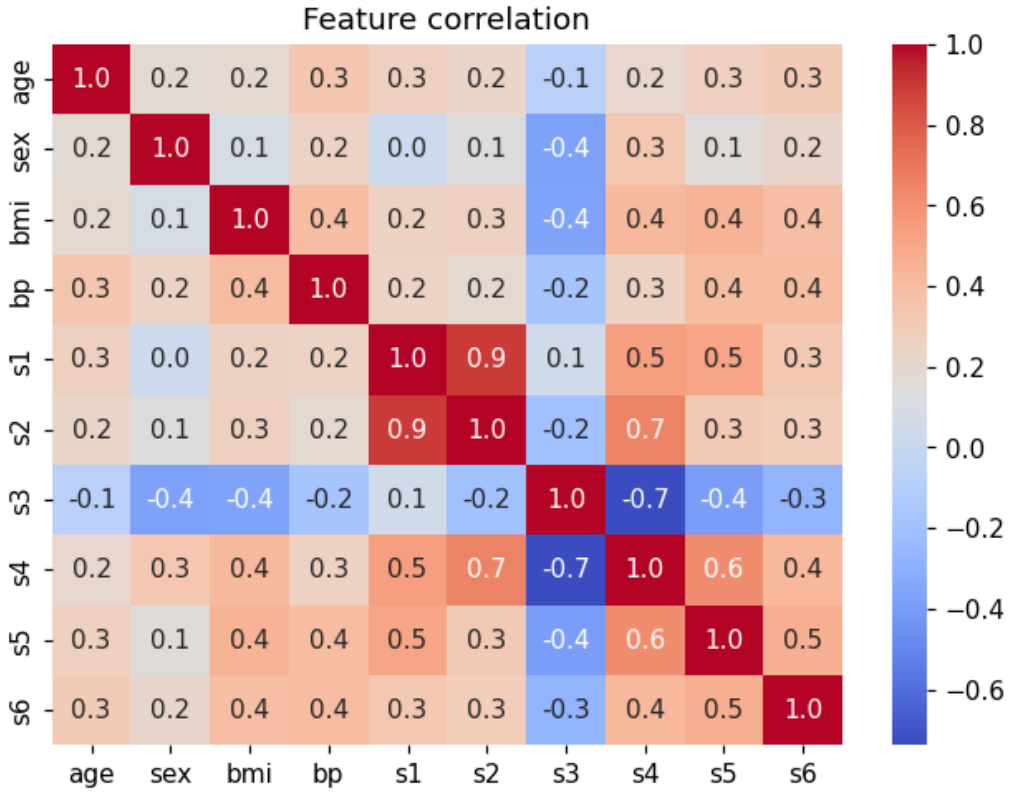
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso

X, y = load_diabetes(return_X_y=True, as_frame=True)

# Multicollinearity check: s1 and s2 are highly correlated
print("Correlation(s1, s2) =", round(X["s1"].corr(X["s2"]), 2))
plt.figure(figsize=(7, 5))
sns.heatmap(X.corr(), annot=True, fmt=".1f", cmap="coolwarm")
plt.title("Feature correlation")
plt.show()
```

Out[1]:

```
Correlation(s1, s2) = 0.9
```



In [2]:

```
# Compare Linear, Ridge and Lasso
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

models = {"Linear": LinearRegression(),
          "Ridge": Ridge(alpha=0.1),
          "Lasso": Lasso(alpha=0.3)}
coefs = pd.DataFrame(index=X.columns)

for name, m in models.items():
    m.fit(X_train, y_train)
    coefs[name] = m.coef_
    print(f"{name:7s} Train R2 = {m.score(X_train, y_train):.3f}   "
```

```
          f"Test R2 = {m.score(X_test, y_test):.3f}")

print("\nCoefficients:")
print(coefs.round(1))
```

Out[2]:

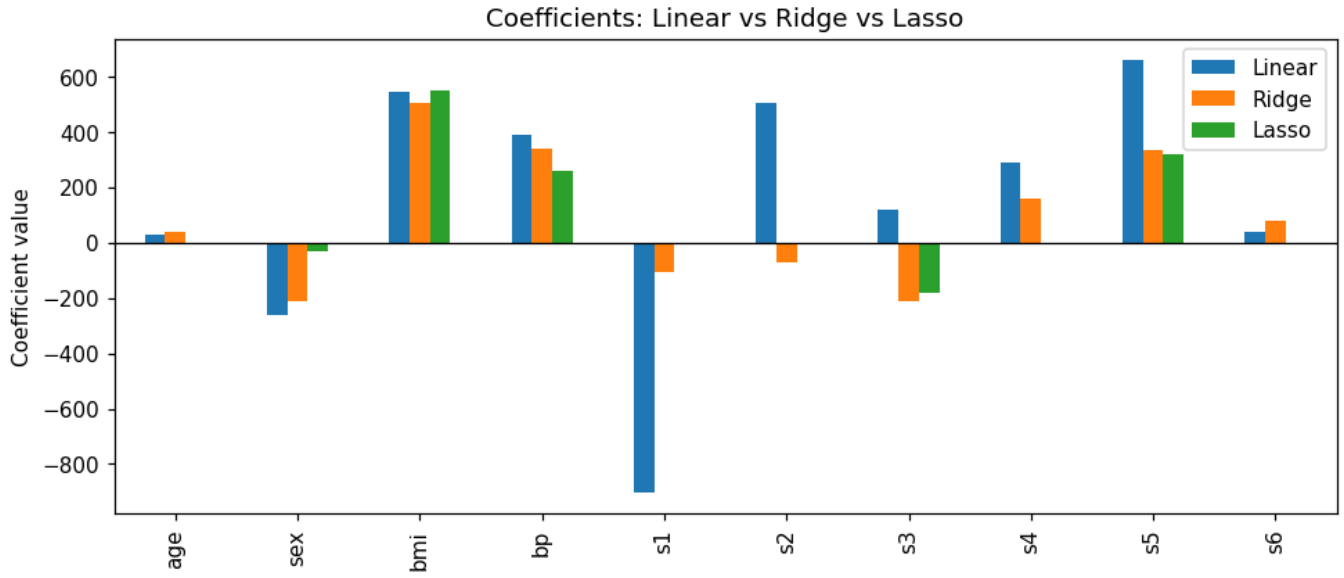
```
 Linear  Train R2 = 0.524   Test R2 = 0.477
 Ridge   Train R2 = 0.517   Test R2 = 0.480
 Lasso   Train R2 = 0.485   Test R2 = 0.473

 Coefficients:
      Linear  Ridge  Lasso
 age    29.3   39.7    0.0
 sex  -261.7 -213.8  -29.7
 bmi   546.3  505.9  552.1
 bp    388.4  341.7  257.1
 s1   -902.0 -108.8   -0.0
 s2    506.8  -70.6   -0.0
 s3    121.2 -211.9 -179.2
 s4    288.0  160.2    0.0
 s5    659.3  332.8  318.5
 s6     41.4   77.7    0.0
```

In [3]:

```
# Coefficient comparison: Ridge shrinks, Lasso sets some to exactly zero
coefs.plot(kind="bar", figsize=(9, 4))
plt.axhline(0, color="black", linewidth=0.8)
plt.title("Coefficients: Linear vs Ridge vs Lasso")
plt.ylabel("Coefficient value")
plt.tight_layout()
plt.show()
```

Out[3]:

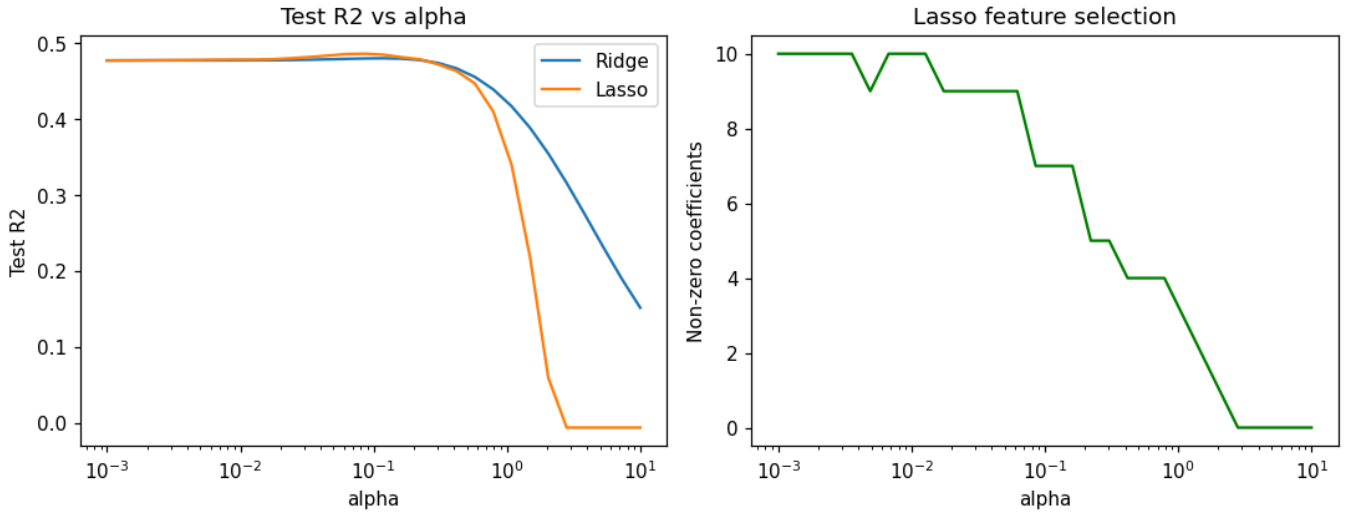


In [4]:

```
# Effect of alpha (regularization strength)
alphas = np.logspace(-3, 1, 30)
ridge_r2, lasso_r2, lasso_nz = [], [], []
for a in alphas:
    ridge_r2.append(Ridge(alpha=a).fit(X_train, y_train).score(X_test, y_test))
    las = Lasso(alpha=a, max_iter=10000).fit(X_train, y_train)
    lasso_r2.append(las.score(X_test, y_test))
    lasso_nz.append(np.sum(las.coef_ != 0))

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].semilogx(alphas, ridge_r2, label="Ridge")
ax[0].semilogx(alphas, lasso_r2, label="Lasso")
ax[0].set_xlabel("alpha"); ax[0].set_ylabel("Test R2"); ax[0].legend()
ax[0].set_title("Test R2 vs alpha")
ax[1].semilogx(alphas, lasso_nz, color="green")
ax[1].set_xlabel("alpha"); ax[1].set_ylabel("Non-zero coefficients")
ax[1].set_title("Lasso feature selection")
plt.tight_layout()
plt.show()
```

Out[4]:



In [5]:

```
# Choose the best alpha automatically with cross-validation
from sklearn.linear_model import RidgeCV, LassoCV

ridge_cv = RidgeCV(alphas=np.logspace(-3, 2, 50)).fit(X_train, y_train)
lasso_cv = LassoCV(cv=5, random_state=42, max_iter=10000).fit(X_train, y_train)
print("Best Ridge alpha:", round(ridge_cv.alpha_, 4),
      "| Test R2:", round(ridge_cv.score(X_test, y_test), 3))
print("Best Lasso alpha:", round(lasso_cv.alpha_, 4),
      "| Test R2:", round(lasso_cv.score(X_test, y_test), 3))
print("Features kept by Lasso:", list(X.columns[lasso_cv.coef_ != 0]))
```

Out[5]:

```
Best Ridge alpha: 0.0543 | Test R2: 0.479
Best Lasso alpha: 0.0053 | Test R2: 0.478
Features kept by Lasso: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's4', 's5', 's6']
```

In [6]:

```
# Overfitting demo: 65 polynomial features but only 60 training rows
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline

X_tr, X_te, y_tr, y_te = train_test_split(X, y, train_size=60, random_state=1)
steps = lambda reg: make_pipeline(PolynomialFeatures(2), StandardScaler(), reg)

for name, reg in [("Linear", LinearRegression()),
                  ("Ridge", Ridge(alpha=100)),
                  ("Lasso", Lasso(alpha=2, max_iter=50000))]:
    m = steps(reg).fit(X_tr, y_tr)
    print(f"{name:7s} Train R2 = {m.score(X_tr, y_tr):.3f}   Test R2 = {m.score(X_te, y_te):.3f}")
```

Out[6]:

```
Linear  Train R2 = 1.000   Test R2 = -68.091
Ridge   Train R2 = 0.622   Test R2 = 0.299
Lasso   Train R2 = 0.777   Test R2 = 0.272
```

In [7]:

```
# Classification with regularization: Breast Cancer data
import warnings
warnings.filterwarnings("ignore")
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression, RidgeClassifier

Xc, yc = load_breast_cancer(return_X_y=True)
Xc_tr, Xc_te, yc_tr, yc_te = train_test_split(
    Xc, yc, test_size=0.3, random_state=42, stratify=yc)

clfs = {
    "Ridge classifier": RidgeClassifier(alpha=10),
    "Logistic (L2 / Ridge)": LogisticRegression(penalty="l2", C=1, max_iter=5000),
    "Logistic (L1 / Lasso)": LogisticRegression(penalty="l1", solver="liblinear", C=0.1),
}
for name, clf in clfs.items():
    pipe = make_pipeline(StandardScaler(), clf).fit(Xc_tr, yc_tr)
    zero = (clf.coef_ == 0).sum()
    print(f"{name:24s} Test accuracy = {pipe.score(Xc_te, yc_te):.3f}  "
```

```
          f"| zero coefficients = {zero} of {Xc.shape[1]}")
```

Out[7]:

```
 Ridge classifier         Test accuracy = 0.942  | zero coefficients = 0 of 30
 Logistic (L2 / Ridge)    Test accuracy = 0.988  | zero coefficients = 0 of 30
 Logistic (L1 / Lasso)    Test accuracy = 0.959  | zero coefficients = 23 of 30
```

Observation / Conclusion

The features s1 and s2 are highly correlated (about 0.9), so ordinary linear regression gives large, opposite-signed coefficients for them (-902 and +507). Ridge shrinks these coefficients (-109 and -71) while keeping all features; Lasso sets age, s1, s2, s4 and s6 to exactly zero (feature selection). In the overfitting demo (65 features, 60 rows) plain linear regression fits the training set perfectly (R2 = 1.0) but fails on the test set (R2 = -68), while Ridge and Lasso give a positive test R2 (0.30 and 0.27). For classification on Breast Cancer, L2 logistic regression scored 98.8% and L1 logistic regression 95.9% while zeroing 23 of 30 coefficients.

Viva Quick Points

- Ridge (L2) adds alpha * sum(w^2): shrinks coefficients but never makes them exactly zero; handles multicollinearity well.
- Lasso (L1) adds alpha * sum(|w|): can set coefficients exactly to zero, so it also performs feature selection.
- Larger alpha -> stronger penalty -> simpler model (more bias, less variance). alpha = 0 is ordinary linear regression.
- Elastic Net combines L1 and L2. In logistic regression the penalty strength is controlled by C = 1/alpha.

Practical 6: Logistic Regression - Binary Classification

Aim: Illustrate the performance of a binary logistic regression classifier using the confusion matrix, precision, recall and F1-score.

Dataset: Breast Cancer Wisconsin dataset (sklearn, 569 samples, 30 features, malignant/benign)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay, accuracy_score,
                             precision_score, recall_score, f1_score,
                             classification_report)

data = load_breast_cancer()
X, y = data.data, data.target
print("Shape:", X.shape)
print("Classes:", list(data.target_names), "-> counts", list(np.bincount(y)))
```

Out[1]:

```
Shape: (569, 30)
Classes: [np.str_('malignant'), np.str_('benign')] -> counts [np.int64(212), np.int64(357)]
```

In [2]:

```
# Split, scale and train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Test accuracy:", round(accuracy_score(y_test, y_pred), 4))
```

Out[2]:

```
Test accuracy: 0.9883
```

In [3]:

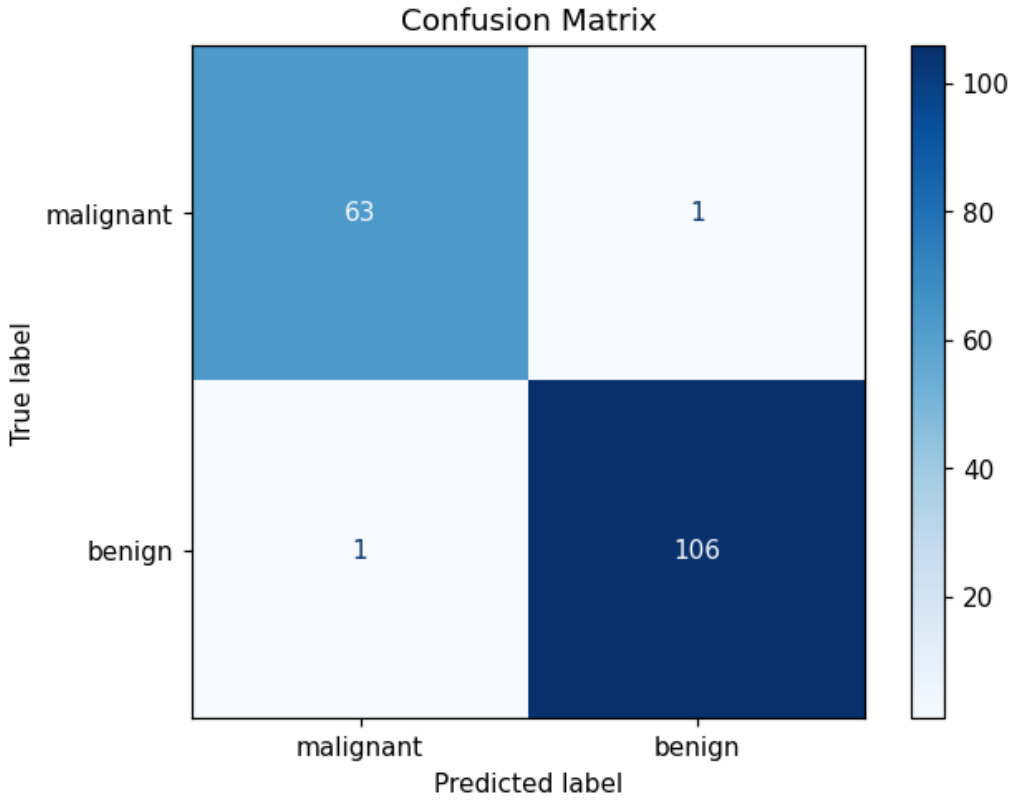
```
# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)

ConfusionMatrixDisplay(cm, display_labels=data.target_names).plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"TN={tn}  FP={fp}  FN={fn}  TP={tp}")
```

Out[3]:

```
[[ 63   1]
 [  1 106]]
TN=63  FP=1  FN=1  TP=106
```



In [4]:

```
# Precision, recall, F1-score
print("Precision:", round(precision_score(y_test, y_pred), 4))
print("Recall   :", round(recall_score(y_test, y_pred), 4))
print("F1-score :", round(f1_score(y_test, y_pred), 4))
print()
print(classification_report(y_test, y_pred, target_names=data.target_names))
```

Out[4]:

```
Precision: 0.9907
Recall   : 0.9907
F1-score : 0.9907

              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        64
      benign       0.99      0.99      0.99       107

    accuracy                           0.99       171
   macro avg       0.99      0.99      0.99       171
weighted avg       0.99      0.99      0.99       171
```

Observation / Conclusion

Logistic regression achieved 98.83% accuracy on the test set. The confusion matrix shows TN = 63, FP = 1, FN = 1, TP = 106. Precision, recall and F1-score are all about 0.99 for the benign class and 0.98 for the malignant class, so the model is both accurate and well balanced.

Viva Quick Points

- Logistic regression applies the sigmoid p = 1/(1 + e^-z) to a linear score z and predicts class 1 if p >= 0.5. It is a classification model despite the name.
- Precision = TP/(TP+FP); Recall = TP/(TP+FN); F1 = 2*P*R/(P+R); Accuracy = (TP+TN)/total.
- Here class 1 = benign. In medical problems, recall of the disease class is usually most important (avoid false negatives).
- Loss function: log-loss (binary cross-entropy).

Practical 7: Support Vector Machine - Linear and Non-linear Kernels

Aim: Use SVM classifiers with linear and non-linear kernels and compare their performance using suitable evaluation metrics.

Dataset: Breast Cancer dataset (kernel comparison) and make_moons (visualising kernels)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer, make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

In [2]:

```
# Compare kernels on Breast Cancer data
rows = []
for k in ["linear", "poly", "rbf", "sigmoid"]:
    svm = SVC(kernel=k, C=1.0, random_state=42)
    svm.fit(X_train, y_train)
    p = svm.predict(X_test)
    rows.append([k, accuracy_score(y_test, p), precision_score(y_test, p),
                 recall_score(y_test, p), f1_score(y_test, p)])

pd.DataFrame(rows, columns=["Kernel", "Accuracy", "Precision", "Recall", "F1"]).round(4)
```

Out[2]:

	Kernel	Accuracy	Precision	Recall	F1
0	linear	0.9825	0.9815	0.9907	0.9860
1	poly	0.8947	0.8560	1.0000	0.9224
2	rbf	0.9766	0.9813	0.9813	0.9813
3	sigmoid	0.9357	0.9528	0.9439	0.9484

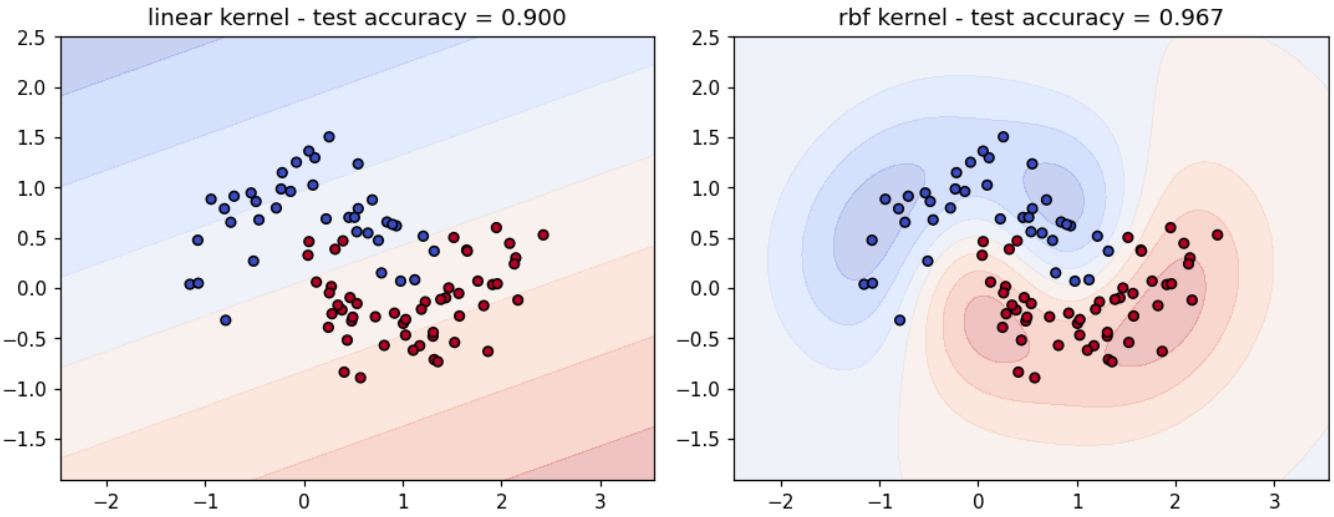
In [3]:

```
# Non-linear data: linear kernel vs RBF kernel
from sklearn.inspection import DecisionBoundaryDisplay

Xm, ym = make_moons(n_samples=300, noise=0.2, random_state=42)
Xm_tr, Xm_te, ym_tr, ym_te = train_test_split(Xm, ym, test_size=0.3, random_state=42)

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
for a, k in zip(ax, ["linear", "rbf"]):
    svm = SVC(kernel=k, C=1.0).fit(Xm_tr, ym_tr)
    DecisionBoundaryDisplay.from_estimator(svm, Xm, ax=a, alpha=0.3, cmap="coolwarm")
    a.scatter(Xm_te[:, 0], Xm_te[:, 1], c=ym_te, cmap="coolwarm", edgecolor="k", s=25)
    a.set_title(f"{k} kernel - test accuracy = {svm.score(Xm_te, ym_te):.3f}")
plt.tight_layout()
plt.show()
```

Out[3]:



In [4]:

```
# Effect of C and gamma for the RBF kernel (moons data)
rows = []
for C in [0.1, 1, 10]:
    for gamma in [0.1, 1, 10]:
        svm = SVC(kernel="rbf", C=C, gamma=gamma).fit(Xm_tr, ym_tr)
        rows.append([C, gamma, round(svm.score(Xm_tr, ym_tr), 3),
                     round(svm.score(Xm_te, ym_te), 3)])
pd.DataFrame(rows, columns=["C", "gamma", "Train acc", "Test acc"])
```

Out[4]:

	C	gamma	Train acc	Test acc
0	0.1	0.1	0.814	0.889
1	0.1	1.0	0.905	0.911
2	0.1	10.0	0.957	0.989
3	1.0	0.1	0.838	0.889
4	1.0	1.0	0.943	0.978
5	1.0	10.0	0.962	0.978
6	10.0	0.1	0.890	0.900
7	10.0	1.0	0.957	0.978
8	10.0	10.0	0.976	0.989

Observation / Conclusion

On the Breast Cancer data the linear kernel (98.25%) and RBF kernel (97.66%) performed best; the polynomial kernel was weaker (89.47%) and sigmoid gave 93.57%. This data is almost linearly separable, so a linear kernel is enough. On the non-linear moons data the RBF kernel separates the classes far better than a linear boundary (see plot). Larger gamma and C fit the training data more closely.

Viva Quick Points

- SVM finds the hyperplane with the maximum margin; support vectors are the points closest to it.
- Kernel trick: computes similarity in a higher-dimensional space without explicitly transforming the data. Linear for linearly separable data; RBF/poly for non-linear data.
- C controls the margin/error trade-off (large C = hard margin, risk of overfitting). gamma (RBF) controls how far one point's influence reaches (large gamma = overfitting).
- Always scale features before using SVM.

Practical 8: K-Means Clustering - Elbow and Silhouette Methods

Aim: Apply K-Means clustering and find the optimal number of clusters using the Elbow and Silhouette methods.

Dataset: Iris dataset (sklearn, 150 flowers, 4 features; labels used only for checking)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.decomposition import PCA

iris = load_iris()
X = StandardScaler().fit_transform(iris.data)       # clustering uses distances -> scale
print("Data shape:", X.shape)
```

Out[1]:

```
Data shape: (150, 4)
```

In [2]:

```
# Elbow method (inertia / WCSS) and Silhouette score for k = 2..10
k_range = range(2, 11)
inertia, sil = [], []
for k in k_range:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertia.append(km.inertia_)
    sil.append(silhouette_score(X, km.labels_))

print(pd.DataFrame({"k": list(k_range), "Inertia": np.round(inertia, 1),
                    "Silhouette": np.round(sil, 3)}).to_string(index=False))
```

Out[2]:

```
 k  Inertia  Silhouette
 2    222.4       0.582
 3    139.8       0.460
 4    114.1       0.387
 5     90.9       0.346
 6     81.5       0.317
 7     72.6       0.320
 8     62.5       0.339
 9     55.1       0.342
10     47.4       0.352
```

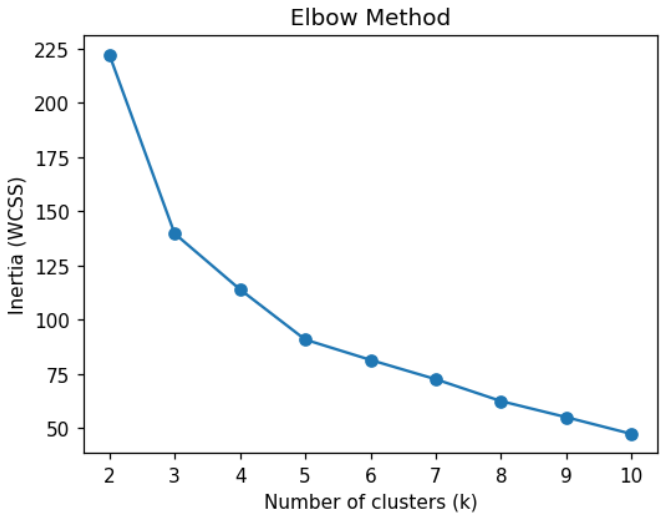
In [3]:

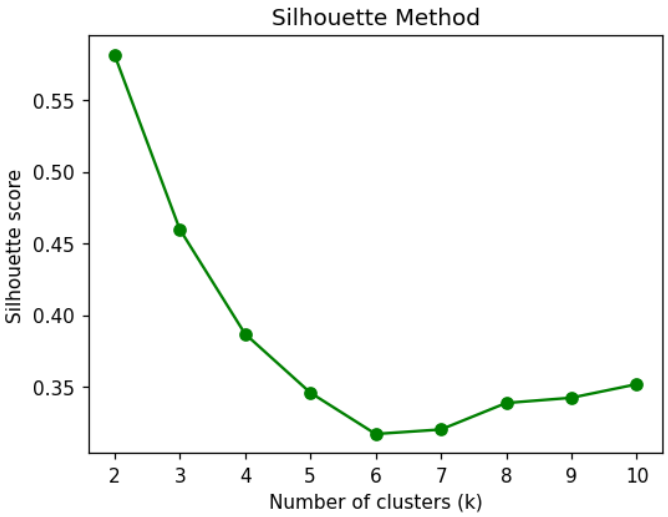
```
# Plot both curves
fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].plot(k_range, inertia, marker="o")
ax[0].set_xlabel("Number of clusters (k)"); ax[0].set_ylabel("Inertia (WCSS)")
ax[0].set_title("Elbow Method")
ax[1].plot(k_range, sil, marker="o", color="green")
ax[1].set_xlabel("Number of clusters (k)"); ax[1].set_ylabel("Silhouette score")
ax[1].set_title("Silhouette Method")
plt.tight_layout()
plt.show()

best_k = list(k_range)[int(np.argmax(sil))]
print("Best k by silhouette score:", best_k)
```

Out[3]:

```
Best k by silhouette score: 2
```





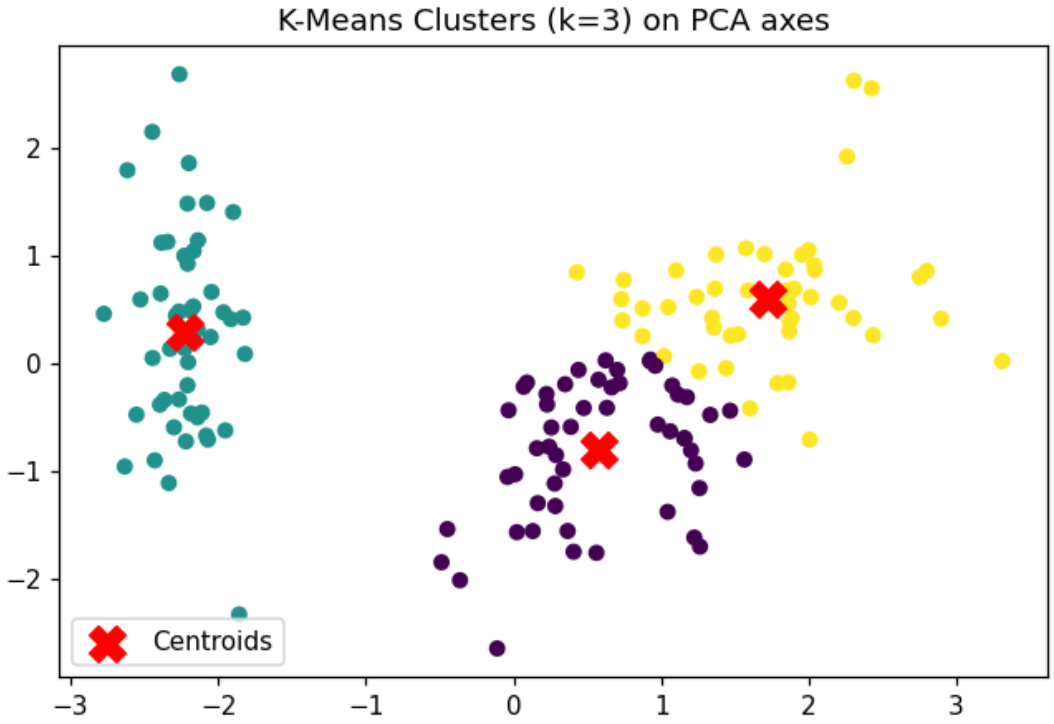
In [4]:

```
# Final model with k = 3 (3 species of iris) and visualisation
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
print("Silhouette score (k=3):", round(silhouette_score(X, km.labels_), 3))
print("Adjusted Rand Index vs true species:",
      round(adjusted_rand_score(iris.target, km.labels_), 3))
print("Cluster sizes:", np.bincount(km.labels_))

X2 = PCA(n_components=2).fit_transform(X)
centers = PCA(n_components=2).fit(X).transform(km.cluster_centers_)
plt.figure(figsize=(7, 4.5))
plt.scatter(X2[:, 0], X2[:, 1], c=km.labels_, cmap="viridis", s=30)
plt.scatter(centers[:, 0], centers[:, 1], c="red", marker="X", s=200, label="Centroids")
plt.title("K-Means Clusters (k=3) on PCA axes")
plt.legend()
plt.show()
```

Out[4]:

```
Silhouette score (k=3): 0.46
Adjusted Rand Index vs true species: 0.62
Cluster sizes: [53 50 47]
```



Observation / Conclusion

The elbow in the inertia curve appears around k = 2-3, and the silhouette score is highest at k = 2 (0.582), then 0.460 at k = 3. Since Iris has 3 species and the elbow is also visible at k = 3, k = 3 was used for the final model (cluster sizes 53/50/47, adjusted Rand index 0.62 against the true species). Two species overlap, which is why the silhouette prefers k = 2.

Viva Quick Points

- K-Means: choose k centroids, assign each point to the nearest centroid, recompute centroids, repeat until convergence.
- Elbow method: plot inertia (within-cluster sum of squares) vs k and pick the 'elbow' where the drop slows down.
- Silhouette score ranges from -1 to 1: higher is better (compact, well-separated clusters).
- Limitations: k must be chosen in advance, sensitive to scaling and initialisation (use k-means++, n_init), assumes round clusters of similar size.

Practical 9: DBSCAN - Density-Based Clustering

Aim: Apply DBSCAN to identify cluster structure and noise points.

Dataset: make_moons (non-spherical clusters) and Iris dataset (sklearn)

Code and Output

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons, load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN, KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score

# Two moon shaped clusters + a few random outliers
X, _ = make_moons(n_samples=300, noise=0.07, random_state=42)
rng = np.random.RandomState(1)
X = np.vstack([X, rng.uniform(-1.5, 2.5, size=(10, 2))])
X = StandardScaler().fit_transform(X)
print("Data shape:", X.shape)
```

Out[1]:

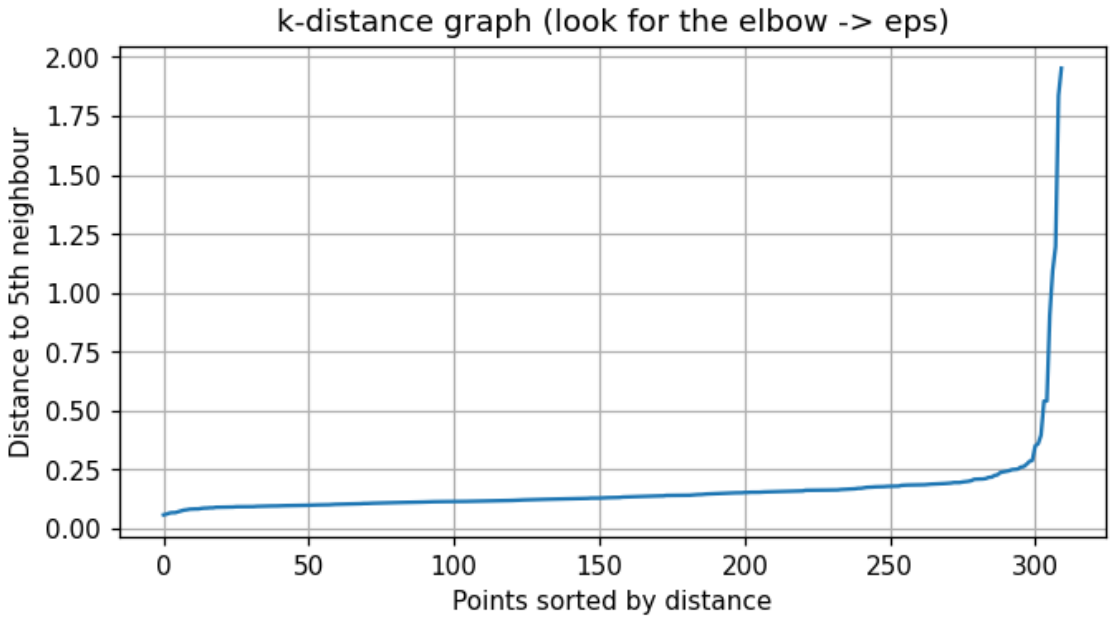
```
Data shape: (310, 2)
```

In [2]:

```
# Choose eps using the k-distance graph (k = min_samples)
min_samples = 5
dist, _ = NearestNeighbors(n_neighbors=min_samples).fit(X).kneighbors(X)
k_dist = np.sort(dist[:, -1])

plt.figure(figsize=(7, 3.5))
plt.plot(k_dist)
plt.xlabel("Points sorted by distance")
plt.ylabel(f"Distance to {min_samples}th neighbour")
plt.title("k-distance graph (look for the elbow -> eps)")
plt.grid(True)
plt.show()
```

Out[2]:



In [3]:

```
# Apply DBSCAN
db = DBSCAN(eps=0.3, min_samples=5).fit(X)
labels = db.labels_                        # -1 means noise

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print("Clusters found :", n_clusters)
print("Noise points   :", list(labels).count(-1))
```

```
print("Core samples   :", len(db.core_sample_indices_))
print("Cluster sizes  :", {int(l): int((labels == l).sum()) for l in set(labels)})
```

Out[3]:

```
Clusters found : 2
Noise points   : 7
Core samples   : 300
Cluster sizes  : {0: 151, 1: 152, -1: 7}
```

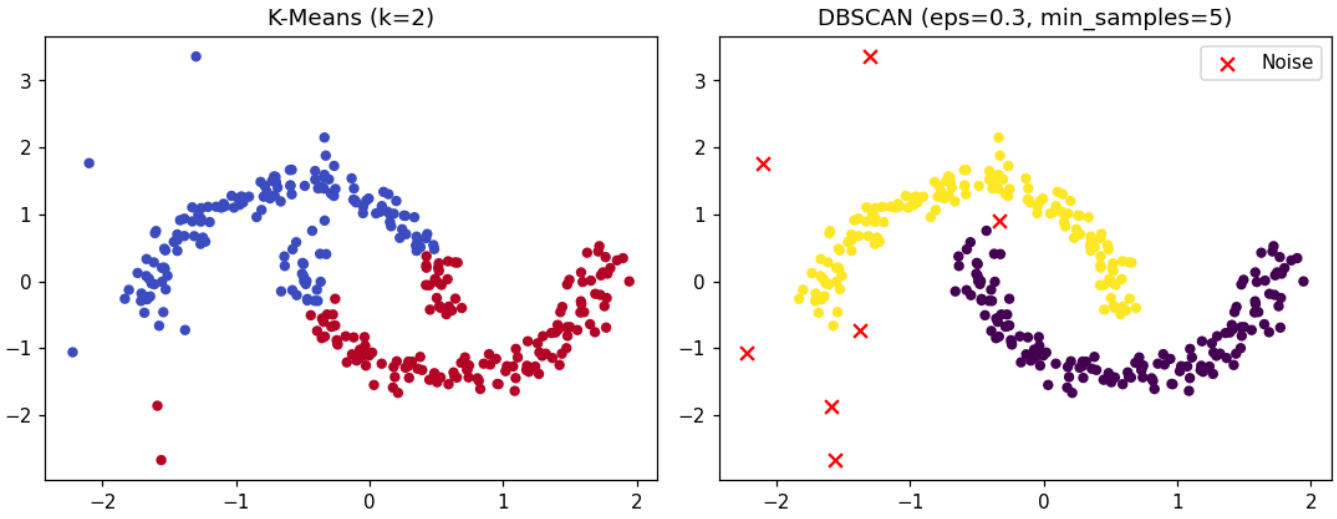
In [4]:

```
# DBSCAN vs K-Means on the same data
km_labels = KMeans(n_clusters=2, n_init=10, random_state=42).fit_predict(X)

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].scatter(X[:, 0], X[:, 1], c=km_labels, cmap="coolwarm", s=20)
ax[0].set_title("K-Means (k=2)")

noise = labels == -1
ax[1].scatter(X[~noise, 0], X[~noise, 1], c=labels[~noise], cmap="viridis", s=20)
ax[1].scatter(X[noise, 0], X[noise, 1], c="red", marker="x", s=50, label="Noise")
ax[1].set_title("DBSCAN (eps=0.3, min_samples=5)")
ax[1].legend()
plt.tight_layout()
plt.show()
```

Out[4]:



In [5]:

```
# DBSCAN on a real dataset (Iris)
Xi = StandardScaler().fit_transform(load_iris().data)
for eps in [0.4, 0.6, 0.8, 1.0]:
    lab = DBSCAN(eps=eps, min_samples=5).fit_predict(Xi)
    n = len(set(lab)) - (1 if -1 in lab else 0)
    print(f"eps = {eps}: clusters = {n}, noise points = {list(lab).count(-1)}")
```

Out[5]:

```
eps = 0.4: clusters = 6, noise points = 66
eps = 0.6: clusters = 2, noise points = 26
eps = 0.8: clusters = 2, noise points = 4
eps = 1.0: clusters = 2, noise points = 3
```

Observation / Conclusion

With eps = 0.3 and min_samples = 5, DBSCAN found the 2 moon-shaped clusters and labelled 7 points as noise (outliers), while K-Means cut the moons incorrectly because it assumes round clusters. On Iris, a small eps gives many clusters and much noise, whereas a large eps merges the data into 2 clusters, so eps must be tuned (k-distance graph).

Viva Quick Points

- DBSCAN needs two parameters: eps (neighbourhood radius) and min_samples (minimum points to form a dense region).
- Core point: has >= min_samples points within eps. Border point: within eps of a core point but not core itself. Noise point: neither (label -1).
- Advantages: no need to specify the number of clusters, finds arbitrary-shaped clusters, detects outliers. Weakness: struggles with clusters of very different densities.
- eps is chosen using the k-distance graph (elbow).

Practical 10: Ensemble Learning - Random Forest and Boosting vs Baseline

Aim: Analyse ensemble learning techniques (Random Forest, AdaBoost, Gradient Boosting) against a baseline Decision Tree.

Dataset: Wine dataset (sklearn, 178 samples, 13 features, 3 classes)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, AdaBoostClassifier,
                              GradientBoostingClassifier, BaggingClassifier)
from sklearn.metrics import accuracy_score, f1_score

data = load_wine()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
print("Shape:", X.shape, "| Classes:", np.unique(y))
```

Out[1]:

```
Shape: (178, 13) | Classes: [0 1 2]
```

In [2]:

```
# Baseline model + ensemble models
models = {
    "Decision Tree (baseline)": DecisionTreeClassifier(random_state=42),
    "Bagging": BaggingClassifier(n_estimators=100, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=100, random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=100, random_state=42),
}

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    pred = m.predict(X_test)
    cv = cross_val_score(m, X, y, cv=5).mean()
    rows.append([name, accuracy_score(y_test, pred),
                 f1_score(y_test, pred, average="macro"), cv])

result = pd.DataFrame(rows, columns=["Model", "Test Accuracy", "Test F1 (macro)",
                                     "5-fold CV Accuracy"]).round(4)
result
```

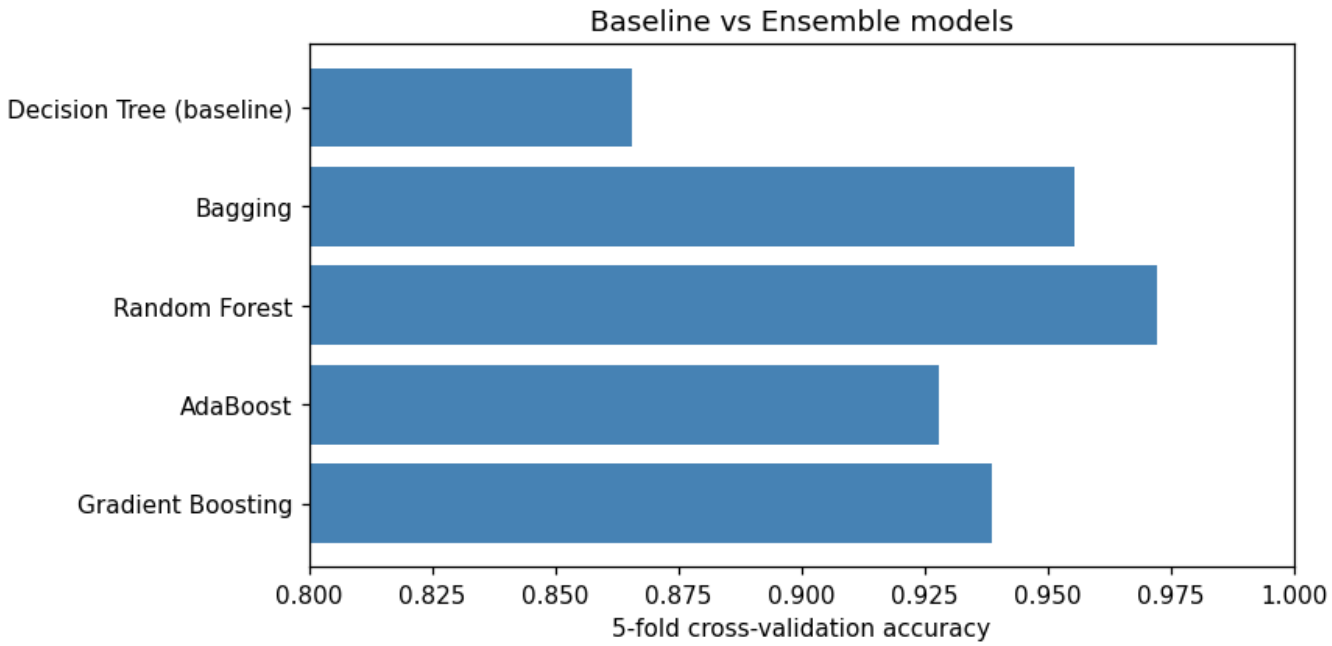
Out[2]:

	Model	Test Accuracy	Test F1 (macro)	5-fold CV Accuracy
0	Decision Tree (baseline)	0.9630	0.9638	0.8654
1	Bagging	1.0000	1.0000	0.9554
2	Random Forest	1.0000	1.0000	0.9721
3	AdaBoost	0.9815	0.9811	0.9278
4	Gradient Boosting	0.9630	0.9652	0.9386

In [3]:

```
# Compare visually
plt.figure(figsize=(8, 4))
plt.barh(result["Model"], result["5-fold CV Accuracy"], color="steelblue")
plt.xlim(0.8, 1.0)
plt.xlabel("5-fold cross-validation accuracy")
plt.title("Baseline vs Ensemble models")
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Out[3]:

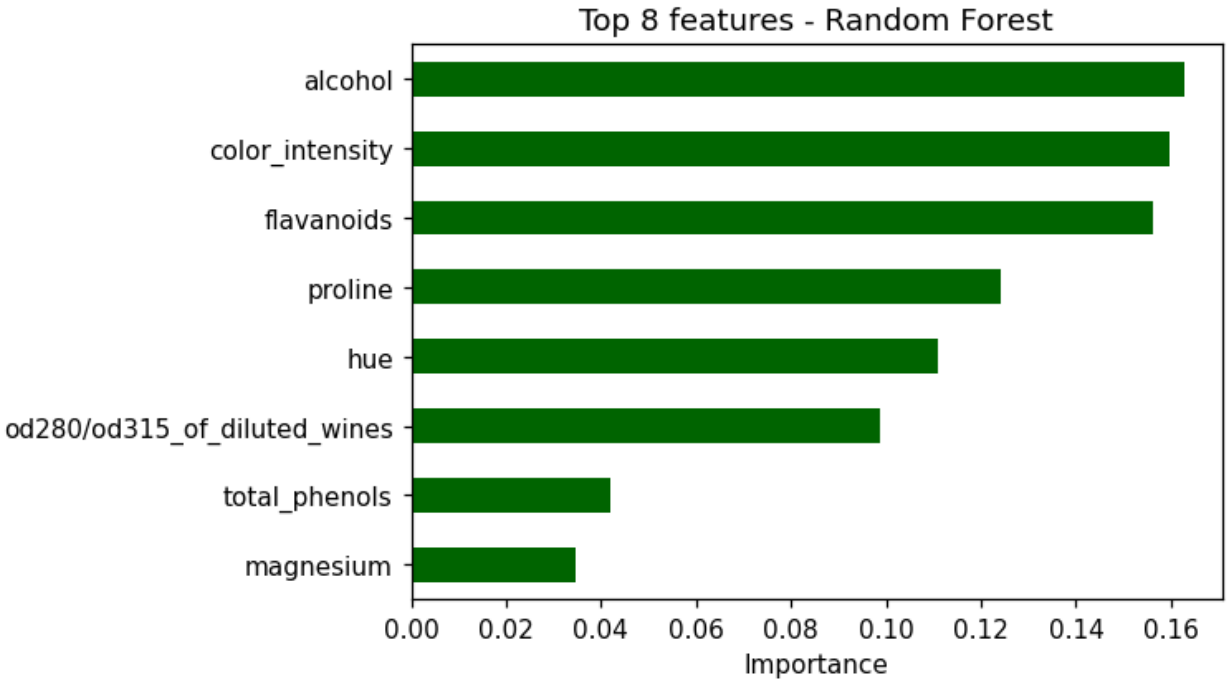


In [4]:

```
# Feature importance from the Random Forest
rf = models["Random Forest"]
imp = pd.Series(rf.feature_importances_, index=data.feature_names).sort_values()

imp.tail(8).plot(kind="barh", figsize=(7, 4), color="darkgreen")
plt.xlabel("Importance")
plt.title("Top 8 features - Random Forest")
plt.tight_layout()
plt.show()
```

Out[4]:



Observation / Conclusion

The baseline Decision Tree reached 96.3% test accuracy and only 86.5% in 5-fold cross-validation, which shows it overfits. All ensembles did better in cross-validation: Bagging 95.5%, AdaBoost 92.8%, Gradient Boosting 93.9% and Random Forest 97.2% (best). Combining many models reduces variance and gives more stable results than a single tree.

Viva Quick Points

- Ensemble = combine many weak/base models to get a stronger one. Bagging/Random Forest: models trained in parallel on bootstrap samples, predictions averaged/voted (reduces variance).

- Boosting (AdaBoost, Gradient Boosting): models trained sequentially, each one focuses on the mistakes of the previous (reduces bias).
- A single decision tree overfits easily; Random Forest adds random feature selection to de-correlate the trees.
- Random Forest gives feature importance; cross-validation gives a more reliable comparison than a single split.

Practical 11: Q-Learning - Grid World

Aim: Evaluate reinforcement learning with the Q-Learning algorithm on a Grid World, focusing on exploration vs exploitation and the optimal policy.

Dataset: 4 x 4 Grid World (custom environment: start, goal, pit)

Code and Output

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 4x4 Grid World          S = start (0,0)   G = goal (3,3) reward +10
#   S . . .               P = pit  (1,1)    reward -10 (episode ends)
#   . P . .               every other step  reward -1
#   . . . .
#   . . . G
SIZE = 4
START, GOAL, PIT = (0, 0), (3, 3), (1, 1)
ACTIONS = ["U", "D", "L", "R"]
MOVES = {"U": (-1, 0), "D": (1, 0), "L": (0, -1), "R": (0, 1)}

def step(state, action):
    r, c = state
    dr, dc = MOVES[action]
    r = min(max(r + dr, 0), SIZE - 1)          # stay inside the grid
    c = min(max(c + dc, 0), SIZE - 1)
    new_state = (r, c)
    if new_state == GOAL:
        return new_state, 10, True
    if new_state == PIT:
        return new_state, -10, True
    return new_state, -1, False
```

In [2]:

```
# Q-learning with epsilon-greedy exploration
def train(episodes=1000, alpha=0.1, gamma=0.9,
          eps_start=1.0, eps_min=0.05, eps_decay=0.995, seed=0):
    rng = np.random.default_rng(seed)
    Q = np.zeros((SIZE, SIZE, 4))               # Q[row, col, action]
    eps = eps_start
    ep_rewards = []
    for ep in range(episodes):
        s, total, done = START, 0, False
        for _ in range(50):                      # max 50 steps per episode
            if rng.random() < eps:
                a = rng.integers(4)              # explore: random action
            else:
                a = np.argmax(Q[s])              # exploit: best known action
            s2, r, done = step(s, ACTIONS[a])
            # Q-learning update rule
            Q[s][a] += alpha * (r + gamma * np.max(Q[s2]) - Q[s][a])
            s, total = s2, total + r
            if done:
                break
        eps = max(eps_min, eps * eps_decay)      # reduce exploration over time
        ep_rewards.append(total)
    return Q, ep_rewards

Q, rewards = train()
print("Training finished. Average reward of last 100 episodes:",
      round(np.mean(rewards[-100:]), 2))
```

Out[2]:

```
Training finished. Average reward of last 100 episodes: 4.41
```

In [3]:

```
# Learned Q-table (rows = state (row, col), columns = actions)
rows = [(r, c) for r in range(SIZE) for c in range(SIZE)]
q_table = pd.DataFrame([Q[r, c] for r, c in rows], index=[str(s) for s in rows],
                       columns=ACTIONS).round(2)
q_table
```

Out[3]:

```
           U      D     L     R
(0, 0)  0.47  -1.63  0.44  1.81
(0, 1)  1.70  -9.99  0.38  3.12
(0, 2)  2.90   4.32  1.67  4.58
(0, 3)  4.40   6.20  2.95  4.29
(1, 0) -2.01   0.12 -2.12 -9.77
(1, 1)  0.00   0.00  0.00  0.00
(1, 2)  1.01   2.06 -7.46  6.19
(1, 3)  4.47   8.00  4.42  5.97
(2, 0) -2.01  -0.97 -1.38  2.42
(2, 1) -6.86   5.12 -0.60  1.02
(2, 2)  0.28   3.87 -0.48  7.85
(2, 3)  5.95  10.00  5.46  7.42
(3, 0) -0.81  -1.02 -0.73  1.58
(3, 1)  0.04   0.89 -0.64  7.68
(3, 2)  1.02   1.39  0.74  9.95
(3, 3)  0.00   0.00  0.00  0.00
```

In [4]:

```python
# Optimal policy (best action in every cell)
def show_policy(Q):
    for r in range(SIZE):
        line = []
        for c in range(SIZE):
            if (r, c) == GOAL:
                line.append("G")
            elif (r, c) == PIT:
                line.append("P")
            else:
                line.append(ACTIONS[np.argmax(Q[r, c])])
        print("  ".join(line))

print("Optimal policy (U=up, D=down, L=left, R=right):")
show_policy(Q)

# Follow the greedy policy from the start
s, path, total = START, [START], 0
while s != GOAL and len(path) < 20:
    s, r, done = step(s, ACTIONS[np.argmax(Q[s])])
    path.append(s)
    total += r
    if done:
        break
print("\nPath:", path)
print("Total reward along the path:", total)
```

Out[4]:

```
Optimal policy (U=up, D=down, L=left, R=right):
R  R  R  D
D  P  R  D
R  D  R  D
R  R  R  G

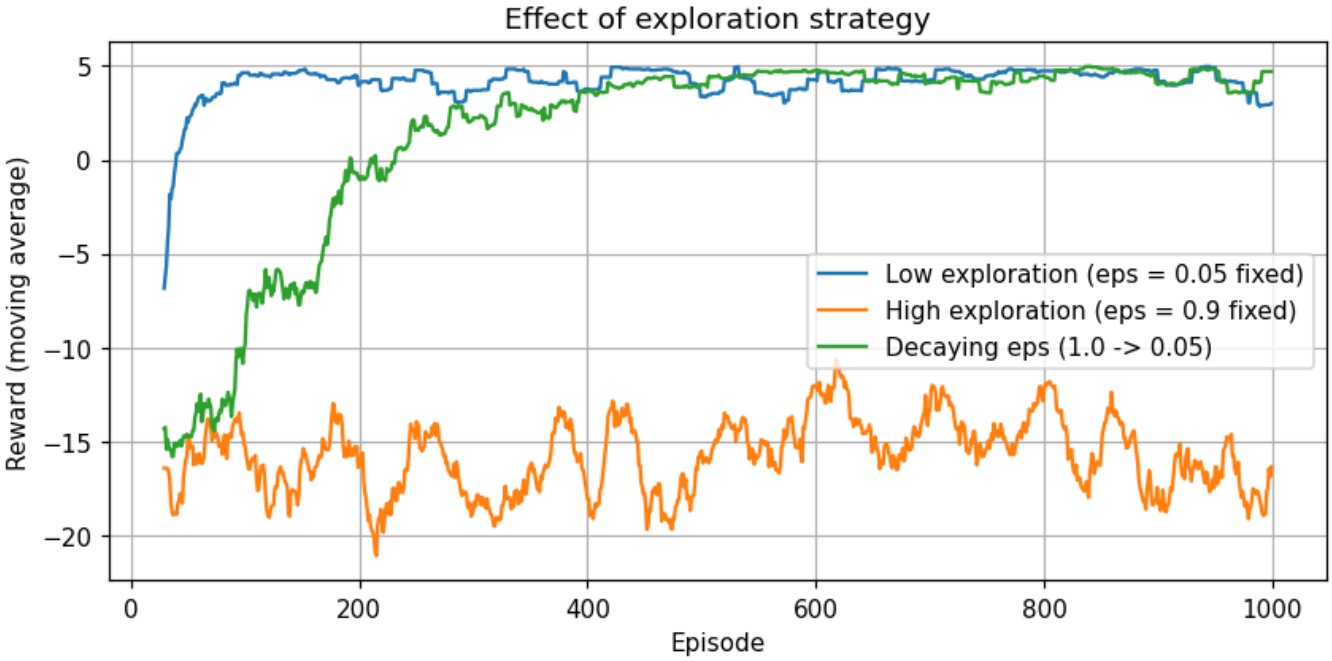
Path: [(0, 0), (0, 1), (0, 2), (0, 3), (1, 3), (2, 3), (3, 3)]
Total reward along the path: 5
```

In [5]:

```python
# Exploration vs exploitation: compare strategies
settings = {
    "Low exploration (eps = 0.05 fixed)": dict(eps_start=0.05, eps_min=0.05, eps_decay=1.0),
    "High exploration (eps = 0.9 fixed)": dict(eps_start=0.9, eps_min=0.9, eps_decay=1.0),
    "Decaying eps (1.0 -> 0.05)": dict(eps_start=1.0, eps_min=0.05, eps_decay=0.995),
}
plt.figure(figsize=(9, 4))
for name, kw in settings.items():
    Qs, rw = train(**kw)
    smooth = pd.Series(rw).rolling(30).mean()
    plt.plot(smooth, label=name)
    print(f"{name:38s} avg reward (last 100 episodes) = {np.mean(rw[-100:]):6.2f}")
plt.xlabel("Episode"); plt.ylabel("Reward (moving average)")
plt.title("Effect of exploration strategy")
plt.legend()
plt.grid(True)
plt.show()
```

Out[5]:

```
Low exploration (eps = 0.05 fixed)     avg reward (last 100 episodes) =   3.86
High exploration (eps = 0.9 fixed)     avg reward (last 100 episodes) = -16.49
Decaying eps (1.0 -> 0.05)             avg reward (last 100 episodes) =   4.41
```



Observation / Conclusion

The agent learned the optimal policy, which goes around the pit and reaches the goal in 6 steps (total reward 5). A fixed low epsilon (0.05) learned reasonably (3.86 average reward), a fixed high epsilon (0.9) kept acting randomly and stayed poor (-16.49), and a decaying epsilon (explore first, exploit later) gave the best result (4.41). Balancing exploration and exploitation is therefore important.

Viva Quick Points

- Q-learning is model-free, off-policy, value-based RL. Update: Q(s,a) = Q(s,a) + alpha * [r + gamma * max Q(s',a') - Q(s,a)].
- alpha = learning rate, gamma = discount factor (importance of future reward), epsilon = probability of exploring a random action.
- Exploration vs exploitation: too little exploration -> may miss the best path; too much -> slow learning. Epsilon decay balances both.
- Optimal policy = pi(s) = argmax_a Q(s,a). Q-table size = states x actions.